DSA STUDY BLUEPRINT SERIES

76. Sliding Window Maximum Queue

Tracking Max Elements in Sliding Windows using Deques

Section 08 • C++ Core Data Structures & Algorithms

Core Concepts & Visual Blueprint

Theoretical models and memory architectures

Key Principles

- **Problem:** Return maximum value in all sliding windows of size K.
- Maintain a decreasing deque of element indices, evicting elements that fall out-of-bounds.

Memory & Execution Blueprint

Nums: [1, 3, -1, -3] (K=3) ⇒ Window [1, 3, -1] Max = **3**

C++ Implementation Reference

Standard code layout and syntax

```
vector maxSlidingWindow(vector& nums, int k) {
    deque dq;
    vector ans;
    for (int i = 0; i < nums.size(); i++) {
        if (!dq.empty() && dq.front() == i - k) dq.pop_front();
        while (!dq.empty() && nums[dq.back()] < nums[i]) dq.pop_back();
        dq.push_back(i);
        if (i >= k - 1) ans.push_back(nums[dq.front()]);
    }
    return ans;
}
```

Algorithmic Complexity & Best Practices

Performance evaluations and critical guidelines

Performance Table

Aspect / Case	Complexity
Time Complexity	O(N)
Space Complexity	O(K) (Window size Deque)

Key Practices & Gotchas

- **Important:** Deque head pointer location always points to current window maximum element index.
- Verify boundary conditions (e.g. empty inputs, single elements).
- Optimize dynamic memory allocations to prevent leaks.

Dry Run & Step-by-Step Execution

Trace analysis on a sample input case

Execution Walkthrough

Let's trace circular queue pointer wrapping when capacity = 3:

Operation	Queue data layout	Pointers [front, rear]	Size count
enQueue(10)	[10, _, _]	[0, 0]	1
enQueue(20)	[10, 20, _]	[0, 1]	2
deQueue()	[_, 20, _]	[1, 1]	1 (wraps correctly)

Interview Variations & Optimization Pathways

Common follow-up problems and optimization strategies

Key Variations & Follow-up Questions

- **Wrap-around indices:** Modulo indexing updates circular buffer limits dynamically.
- **Deque mechanics:** Doubly ended queues allow fast O(1) front and rear pushes and pops.
- **Related LeetCode Problems:** #239 Sliding Window Maximum, #622 Design Circular Queue, #225 Implement Stack using Queues.